

Step 4 — A Real Client That Connects and Uses All Three — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — the direct way

```
# client.py
import asyncio
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

async def main():
    server_params = StdioServerParameters(command="python", args=["server.py"])
    async with stdio_client(server_params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()

            tools = await session.list_tools()
            print([t.name for t in tools.tools])

            result = await session.call_tool("add_note", {"text": "Buy milk and eggs"})
            print(result)

asyncio.run(main())
```

This proves a client can connect and call one tool. It's missing the Resource and Prompt calls, and any handling for something going wrong.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — the full loop, all three building blocks

```
# client.py
import asyncio
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

async def main():
    server_params = StdioServerParameters(command="python", args=["server.py"])
    async with stdio_client(server_params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()

            tools = await session.list_tools()
            print("Tools:", [t.name for t in tools.tools])

            resources = await session.list_resources()
            print("Resources:", [r.uri for r in resources.resources])

            prompts = await session.list_prompts()
            print("Prompts:", [p.name for p in prompts.prompts])

            add_result = await session.call_tool("add_note", {"text": "Buy milk at store"})
            print("add_note result:", add_result)

            search_result = await session.call_tool("search_notes", {"query": "milk"})
            print("search_notes result:", search_result)

            resource_result = await session.read_resource("notes://all")
            print("notes://all:", resource_result)

            prompt_result = await session.get_prompt(
                "summarize_notes", {"start_date": "2026-09-01", "end_date": "2026-09-01"}
            )
            print("summarize_notes prompt:", prompt_result)
```

```
asyncio.run(main())
```

Expected output (shapes will vary slightly by SDK version, but every call succeeds and returns something readable):

```
Tools: ['add_note', 'search_notes']
Resources: ['notes://all']
Prompts: ['summarize_notes']
add_note result: CallToolResult(content=[TextContent(text='Note added with id 1.')]
search_notes result: CallToolResult(content=[TextContent(text="['Buy milk and egg
notes://all: ReadResourceResult(contents=[TextResourceContents(text='[1] Buy milk
summarize_notes prompt: GetPromptResult(messages=[...prompt text with the 2 dates
```

Difference from Basic: every one of the 4 things the server offers gets discovered (`list_tools` / `list_resources` / `list_prompts`) and used (`call_tool` twice, `read_resource` , `get_prompt`) in one run — this is the "full loop" the Charter promised, not just one tool call.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — connection failures and a checked `isError` flag

```
# client.py
import asyncio
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

async def main():
    server_params = StdioServerParameters(command="python", args=["server.py"])
    try:
        async with stdio_client(server_params) as (read, write):
            async with ClientSession(read, write) as session:
                await session.initialize()

                tools = await session.list_tools()
                print("Tools:", [t.name for t in tools.tools])

                resources = await session.list_resources()
                print("Resources:", [r.uri for r in resources.resources])

                prompts = await session.list_prompts()
                print("Prompts:", [p.name for p in prompts.prompts])

                add_result = await session.call_tool("add_note", {"text": "Buy mi
                print("add_note result:", add_result)

                search_result = await session.call_tool("search_notes", {"query":
                print("search_notes result:", search_result)

                resource_result = await session.read_resource("notes://all")
                print("notes://all:", resource_result)

                prompt_result = await session.get_prompt(
                    "summarize_notes", {"start_date": "2026-09-01", "end_date": "
                )
                print("summarize_notes prompt:", prompt_result)

    # deliberately trigger Step 2's validation, and check the flag
```

```

        bad_result = await session.call_tool("add_note", {"text": "  "})
        if bad_result.isError:
            print("add_note correctly rejected empty text:", bad_result)
        else:
            print("WARNING: empty note was not rejected -- check Step 2's
except Exception as e:
    print(f"Could not start or connect to the notes server: {e}")

asyncio.run(main())

```

Expected behavior: with a working `server.py`, every line above runs and the empty-text call prints the "correctly rejected" line. Temporarily rename `server.py` to something else and rerun `client.py` — instead of a raw Python traceback, you get one clear line: `"Could not start or connect to the notes server: ..."`. Rename it back before moving on.

Approach 2 — never hardcoding what the server offers

```

async def call_every_tool_with_sample_args(session, tools_result):
    """Illustrates discovery-driven calling instead of hardcoded tool names --
    not something you need for this project's fixed 2 tools, but the shape
    a real agent's MCP client actually needs once a server's tool list can
    change without the client being rewritten."""
    sample_args = {
        "add_note": {"text": "Sample note from discovery loop"},
        "search_notes": {"query": "Sample"},
    }
    for tool in tools_result.tools:
        if tool.name in sample_args:
            result = await session.call_tool(tool.name, sample_args[tool.name])
            print(f"{tool.name} ->", result)

```

Expected behavior: functionally the same output as calling `add_note` and `search_notes` by name directly, for this project's 2 fixed tools. The difference only shows up if `server.py` later gains a third tool — Approach 1's client would keep working unmodified and simply never call the new one; a client that assumed "there are exactly 2 tools, always" somewhere in its logic (for example, an `if/elif` chain listing them by name instead of looping over whatever `list_tools()` returns) would need a code change just to keep working, which defeats the entire point of MCP's discovery model.

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate proves the happy path works. Approach 1 adds the two failure-handling habits (a wrapped connection attempt, a checked `isError` flag) that turn "the loop works when everything goes right" into "the loop tells you clearly when something doesn't." Approach 2 is a design habit, not a bug fix — it's the

difference between a client that happens to work today and one that stays correct as the server it depends on grows.

Which one should you actually write? Approach 1's full script is what belongs in this project's `client.py` — the failure handling is cheap to add and turns a confusing traceback into a one-line, readable failure, exactly like every earlier project's `check_api_key_at_startup()` -style checks. Approach 2's discovery-driven pattern is worth understanding now, even for this project's fixed 2 tools, because it's the exact shape a real agent's MCP integration needs — and recognizing "don't hardcode what a server offers" as the same idea underneath both is what actually makes this project a genuine preview of building an MCP *client* properly, not just a script that happens to call two known tool names.